



HIDDEN TECH DEBT IN AGENTS

In 2015, Google warned us:

**"The model is
just a **tiny box.**"**

**11 years later, the SAME warning hits AI Agents —
and 99% of teams miss it.**

What production agents really look like →



Akhila

AI & Tech



Follow me

THE REAL JOB

The model is the **small box.** Everything else is the JOB.

The 9 components where production agents actually live and die.



Beginner-friendly, with analogies



Concrete next steps per layer



The runtime is the new ML infrastructure



Akhila

AI & Tech



Follow me

The 2015 Lesson That Predicted Today



A paper that named our profession

In 2015, Google engineers published a NeurIPS paper: "**Hidden Technical Debt in Machine Learning Systems.**" The point was brutal:

- ⊗ The model code is a rounding error
- ⊗ Everything around it — pipelines, monitoring, serving — is where systems live, fail & accrue debt



It spawned MLOps

And every "ML Engineer" job title you scroll past today.

The same pattern is happening RIGHT NOW with AI Agents.



Akhila
AI & Tech

 **Follow me**

THE REMIX

Same Diagram. New Decade.

02



"Everything else" got rewritten



In 2015 it was

Feature stores · data pipelines · drift monitoring · model serving.



In 2026 it is

Orchestrator · tool layer · memory · sandbox runtime · tracing · eval harness · guardrails.

The model is rented. Your infrastructure is your IP.



Akhila

AI & Tech



Follow me

Markdown Configs

Your prompts are the real source code

Prompts, skills, sub-agent definitions, slash commands are NOT docs. When you "build an agent," 80% of the work is editing prose: system prompts, tool descriptions, few-shot examples, role specs.

- ✓ Version control every prompt (prompts/main_agent.md)
- ✓ Diff prompt changes in PR reviews like code
- ✓ Tag versions & tie them to eval scores
- ✓ Roll out behind feature flags

Markdown configs deserve the same discipline as code.



Akhila

AI & Tech



Follow me

The Model Layer



"How many models, and which fires when?"

Analogy 12 calls on a frontier model $\approx$ \$0.40. Move 60% to a small model $\approx$ \$0.18. Multiply by 10,000 daily users $\rightarrow$ a 6-figure annual decision.



A real setup has 3 things

A router (often non-LLM) · a model fleet (small/mid/large + specialists) · a fallback chain for when Provider A is down.

Use LiteLLM / OpenRouter / AI Gateway — or switching = a 6-week project.



Akhila
AI & Tech



Follow me

The Agent Harness



Open any production agent — it's a state machine

ReAct

Reason → Act → Observe → Repeat. Robust, slightly token-heavy.
The right default.

Plan-and-Execute

Planner makes the plan, executor runs it. Cheap, but brittle when plans change.

Reflexion / Self-Critique

A critic pass after each step. Expensive, but raises quality on hard tasks.

Guardrails it MUST own: step limit · \$ circuit breaker · smart retries.



Akhila

AI & Tech

 **Follow me**

The Tool Layer



The agent doesn't see tools — it sees descriptions

Analogy One team's agent burned its whole monthly API budget in an afternoon — it re-called a failing API 47 times. The bug wasn't the API. It was the tool description.

<> Every tool has 3 parts

Schema (in/out) · implementation (the code) · description (the natural-language hint — the highest-leverage piece).

15 narrow tools beat 1 mega-tool — accuracy collapses at 15–25 tools.



Akhila
AI & Tech



Follow me

Memory

Not one vector DB — a stack of subsystems

Tier 1 — Scratchpad

Current conversation + recent tool outputs. Solve: summarization & pruning.

Tier 2 — Session

Rolling summary + last N raw turns. Solve: consistency — this is where bugs live.

Tier 3 — Long-term

Persists across sessions/users. Solve: retrieval without noise.

The 3 AM bug is session summaries drifting from raw history.



Akhila
AI & Tech

 **Follow me**

Serving Infrastructure



An agent run is NOT a web request

Analogy A web request is short, stateless, uniform. An agent run is long, stateful, heterogeneous — and spends most of its life IDLE, waiting on tools, webhooks & human approval.

5 questions your serving design must answer:

- ✓ Invocation model — sync, async, or event-triggered?
- ✓ Where does state live — in-process or external DB?
- ✓ Scaling — fan-out, multi-tenant, long-lived?
- ✓ Can serving talk to your tracing?
- ✓ Cost — are you billed during idle time?



Akhila

AI & Tech



Follow me

Observability & Tracing



Don't look at the answer — look at the 14 steps before it

⊗ Step 7: search returned empty → model fabricated a result

⊗ Step 12: JSON parser silently dropped a field

⊗ Step 4–5: orchestrator looped silently

🕒 5 metrics that matter

Task success rate (binary) · cost per successful outcome · P95 latency · tool error rate · avg steps per task.

Tools: LangSmith · Langfuse · W&B Traces · Logfire · Opik.



Akhila

AI & Tech

 **Follow me**

The Evaluation Harness



No evals = a demo with vibes attached

🎯 4 types you'll need

Reference-based (golden in/out) · reference-free (LLM-as-judge) · trajectory (score the path) · end-to-end (real-user shadow mode).



Beware "eval set rot" — refresh monthly with sampled real traffic.

The eval suite is THE deliverable. The agent just scores well on it.



Akhila

AI & Tech



Follow me

Guardrails & Safety

Threat-model your agent — what can go wrong?

-  Prompt injection in the user message
-  Prompt injection in tool output
-  PII leaking from memory across users
-  Runaway cost from malicious 30-step loops
-  Off-policy financial / medical / legal advice

3 engineering rules

Rules in code, not prompts · human-in-the-loop on irreversible actions · guardrails are a design constraint, not a last-minute feature.

Defense in depth — Swiss cheese: the holes never line up.



Akhila
AI & Tech

 **Follow me**

Your agent stack **in 2026**



Markdown Configs — the source code



Model Layer — router + fleet



Agent Harness — state machine



Tool Layer — schema + description



Memory — 3 tiers



Serving Infra — 5 design questions



Observability — trace + monitor



Eval Harness — THE deliverable



Guardrails — defense in depth

The model is the small box. Everything else is the JOB.



Akhila

AI & Tech



Follow me

TAKEAWAY

If You're Building Agents in 2026

- ✓ The model is the small box — don't optimize it first
- ✓ The runtime is the new ML infrastructure — pick it deliberately
- ✓ Own the whole picture, or own the seams between components



The role has a name

AI Systems Engineer. Same shape as MLOps in 2015 — new decade, new components.

Credit Based on Miguel Otero Pedrido's brilliant article on The Neural Maze — go read the full piece for deeper insights.



Akhila

AI & Tech



Follow me

FOUND THIS USEFUL?

Share it with
your team

Repost

Save

Comment

Follow Akhila for more such insights →



Akhila

AI & Tech



Follow me